Multi-Site Enterprise Network Topology & Automated Infrastructure

Detailed Technical Implementation & Verification Report

Author: Saif Al-Deen Al-Sayde

Environment: EVE-NG & VMware Workstation

Domain: test.local

Date: August 2026

1. Executive Summary

This project demonstrates the end-to-end design, implementation, and verification of a hybrid multi-site enterprise network infrastructure. It integrates core multi-layer switching, WAN edge routing, Active Directory domain management, and automated network configuration backups. Key technical components include dynamic IP addressing via multi-scope DHCP, GRE site-to-site tunneling, containerized Linux web services, and an autonomous Docker/Netmiko automation pipeline scheduled via system cron jobs.

2. Network Topology & Architecture

The network topology spans two distinct core sites interconnected via a site-to-site tunnel, providing structural redundancy, network isolation, and high scalability.

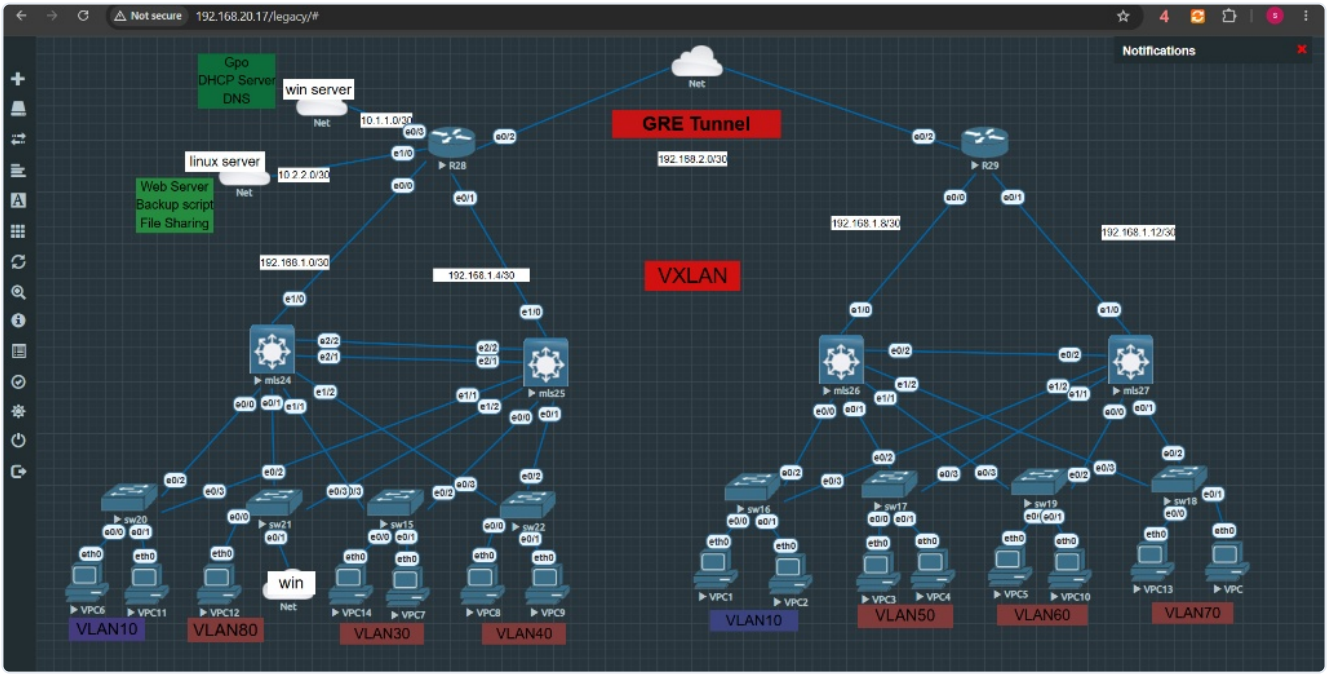

Figure 2.1: Full Multi-Site Network Topology Designed in EVE-NG

Site 1 (Main Site) Architecture

- **Core Layer:** Redundant Cisco Multi-Layer Switches (`mls24`, `mls25`) with cross-link aggregation.

Site 2 (Remote Site) & Inter-Site Routing

- **Core Layer:** Redundant Multi-Layer Switches (`mls26`, `mls27`).

- **Access Layer:** VLAN-segmented access switches:
  - VLAN 10 (192.168.10.0/24) - End Devices / Users
  - VLAN 20 (192.168.20.0/24) - Server Infrastructure
  - VLAN 30 (192.168.30.0/24) - Operations
  - VLAN 40 (192.168.40.0/24) - Engineering
  - VLAN 80 (192.168.80.0/24) - Domain Workstations
- **Edge Gateway:** Router R28 providing NAT and WAN routing.
- **Access Layer VLANs:**
  - VLAN 10 (192.168.10.0/24) - Remote Users
  - VLAN 50 (192.168.50.0/24) - Management
  - VLAN 60 (192.168.60.0/24) - Operations
  - VLAN 70 (192.168.70.0/24) - Guest Network
- **Edge Gateway:** Router R29 terminating inter-site traffic.
- **GRE Tunnel:** Subnet `192.168.2.0/30` established between R28 and R29 for direct site-to-site communication.

3. Server Infrastructure & Centralized Services

3.1 Active Directory, Multi-Scope DHCP, & DNS Configuration

Centralized identity, domain policies, and dynamic IP addressing are delivered by a Windows Server instance operating as the Primary Domain Controller (`pdc.test.local`).

- **Active Directory Domain Services (AD DS):** Controls access and authentication across domain-joined machines in `test.local`.
- **Multi-Scope DHCP Server:** Active scopes configured for each VLAN (e.g., `192.168.10.0`, `192.168.20.0`, `192.168.80.0`) with default gateway and DNS options.
- **DNS Server:** Resolves internal hostnames across servers and infrastructure components.

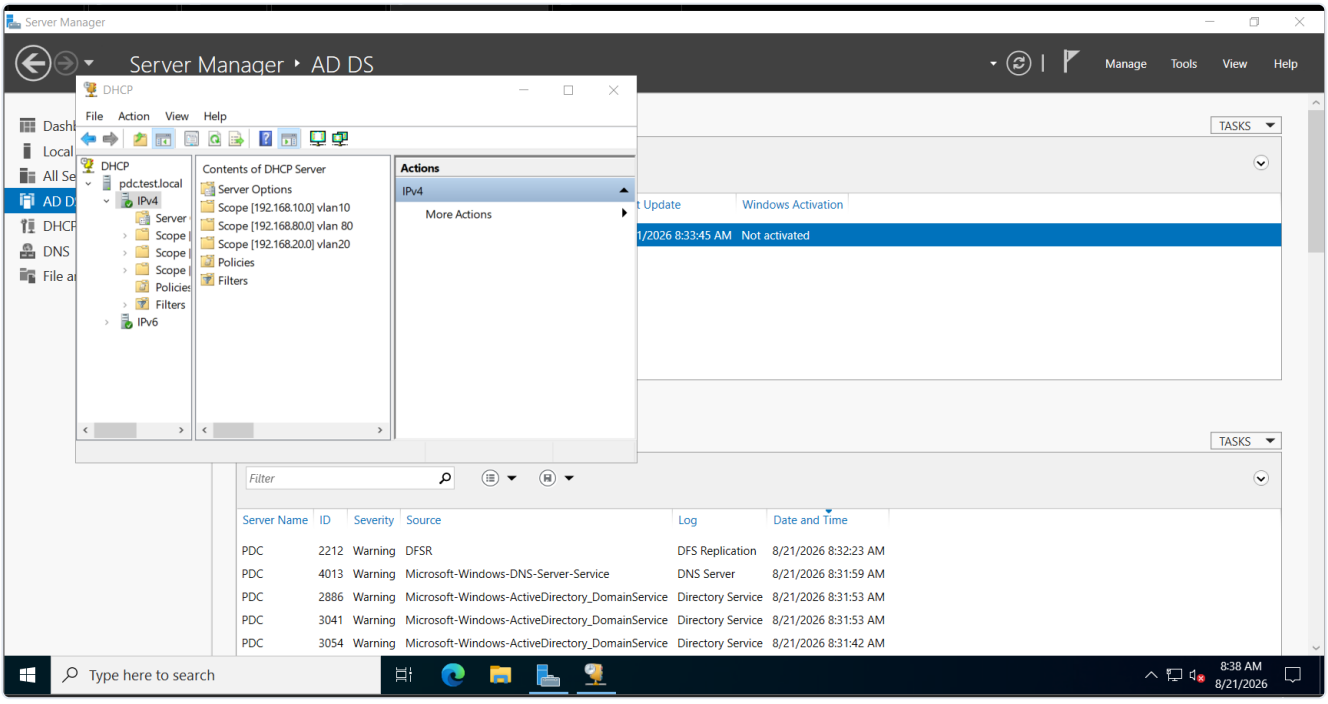

Figure 3.1: Windows Server Manager - Active Directory & Multi-Scope DHCP Configuration

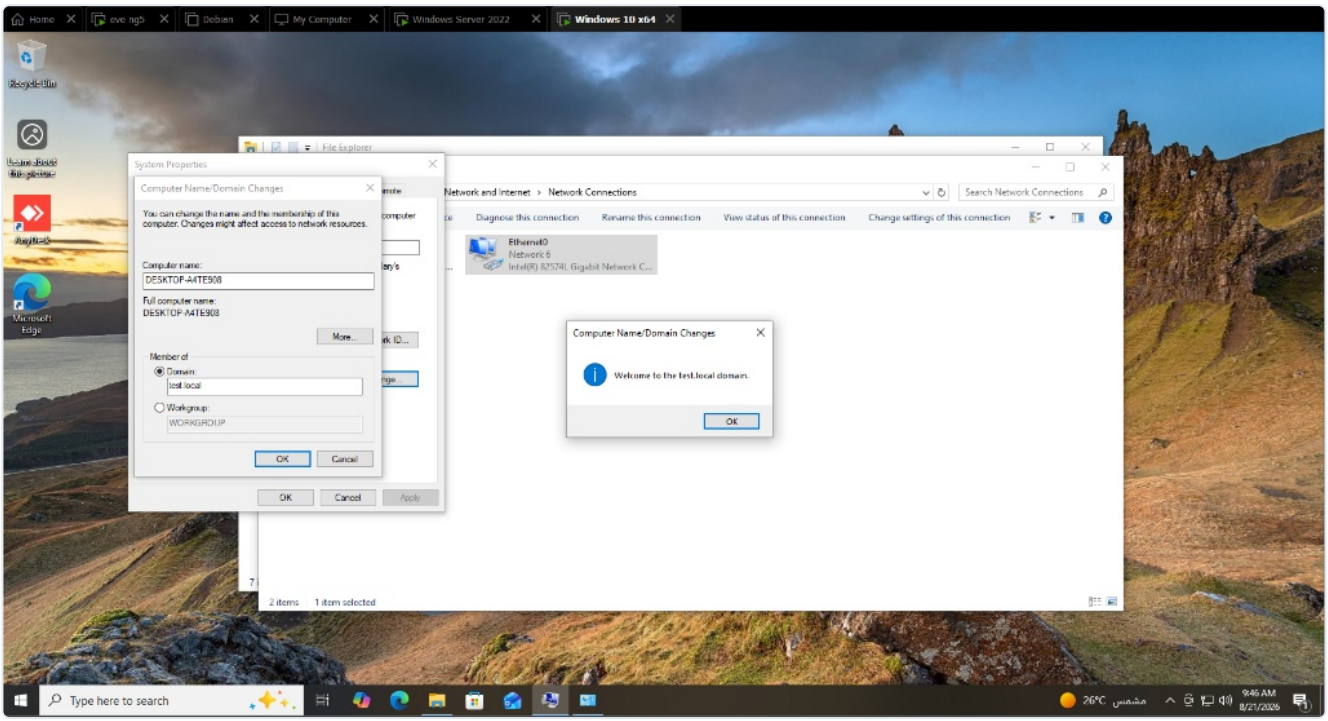

Figure 3.2: Workstation Joined to the `test.local` Domain

3.2 Hosted Web Applications & Service Forwarding

A Linux Debian host runs web application services on non-standard ports (`8080` and `8090`). Port forwarding rules enable internal accessibility across the topology.

Figure 3.3: Web Application Hosted on Port 8080

Figure 3.4: Web Application Hosted on Port 8090

4. Network Automation & Backup Solution

To automate recurring routine management tasks, a containerized automation engine was developed and deployed on the Linux host.

4.1 Containerized Netmiko Script Engine

- **Dockerization:** Container image (`saifaldeenalsayde/lastversionbackup`) embedding Python, Netmiko, and data-formatting libraries.
- **Configuration Retrieval:** Connects via SSH to network routers (`R1`, `R2`), retrieves running configurations, and saves timestamped backups.

- **Excel Data Parsing:** Parses interface status summaries into structured Excel workbooks (e.g., `int_br_to_r1.xlsx`).
- **Exception & Error Handling:** Catches timeouts and connection errors, logging detailed error timestamps into `ssh_failures.log`.

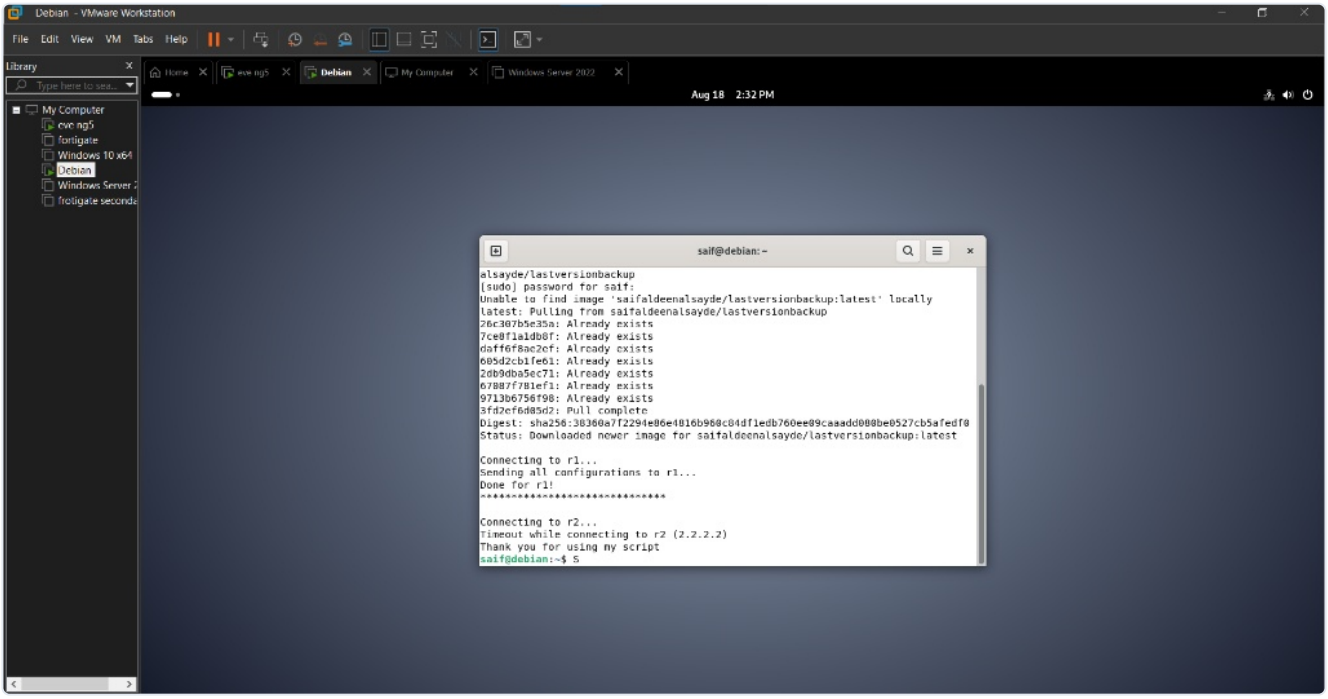

Figure 4.1: Debian Host Execution of Network Automation Docker Container

4.2 System Cron Job Scheduling

Automated execution is configured via the Linux system crontab, ensuring hands-free recurring execution.

```
# Crontab schedule entry (Runs every Monday at 02:00 AM)
0 2 * * 1 docker run -it -v /home/saif/output11:/app/output saifaldeenalsayde/lastversionbackup
```

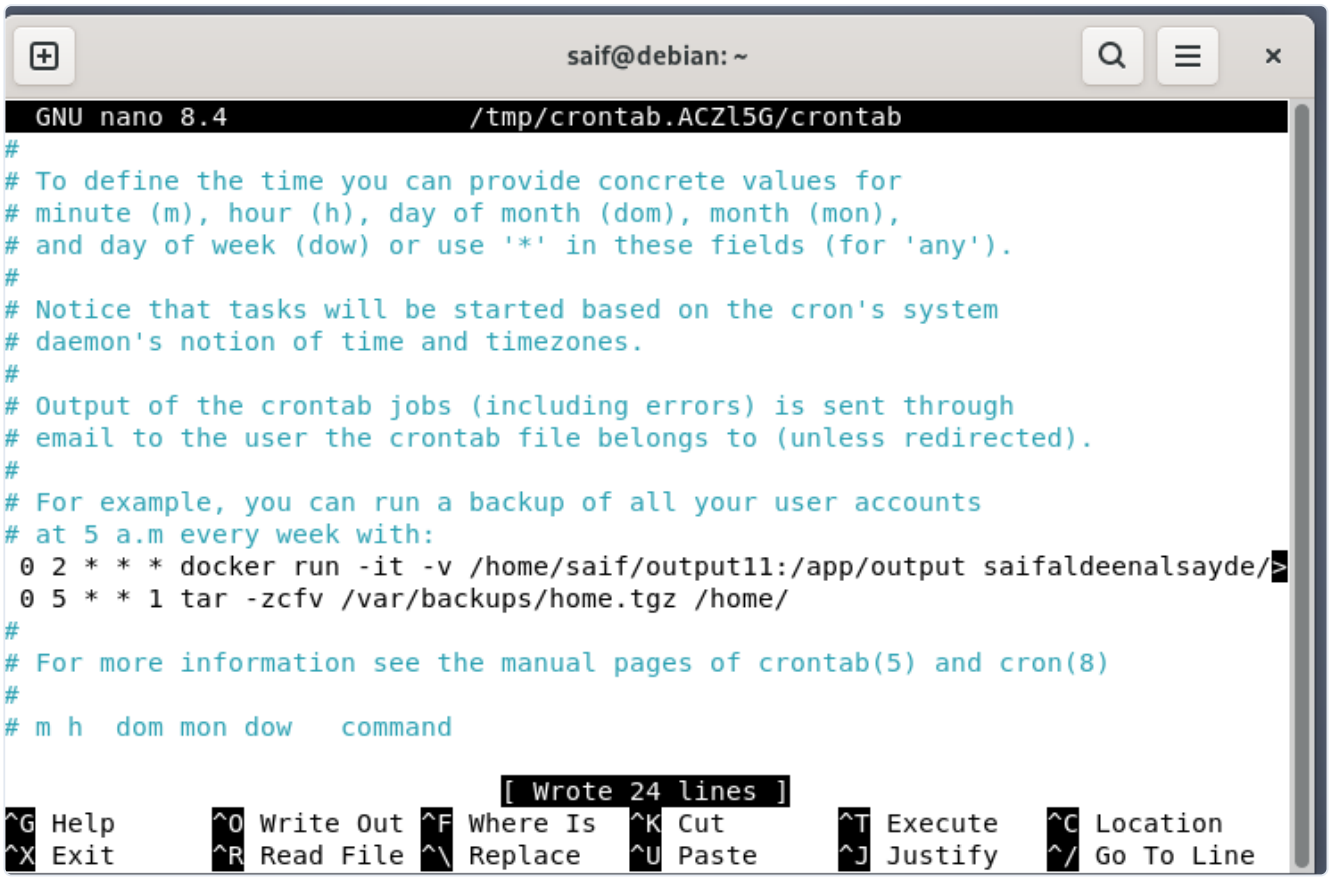

Figure 4.2: Linux System Crontab Scheduling Autonomous Backup Execution

5. Verification, Error Handling & Execution Results

5.1 Client DHCP & Internet Reachability Testing

End devices successfully complete the DHCP DORA process and establish outbound reachability to external public destinations.

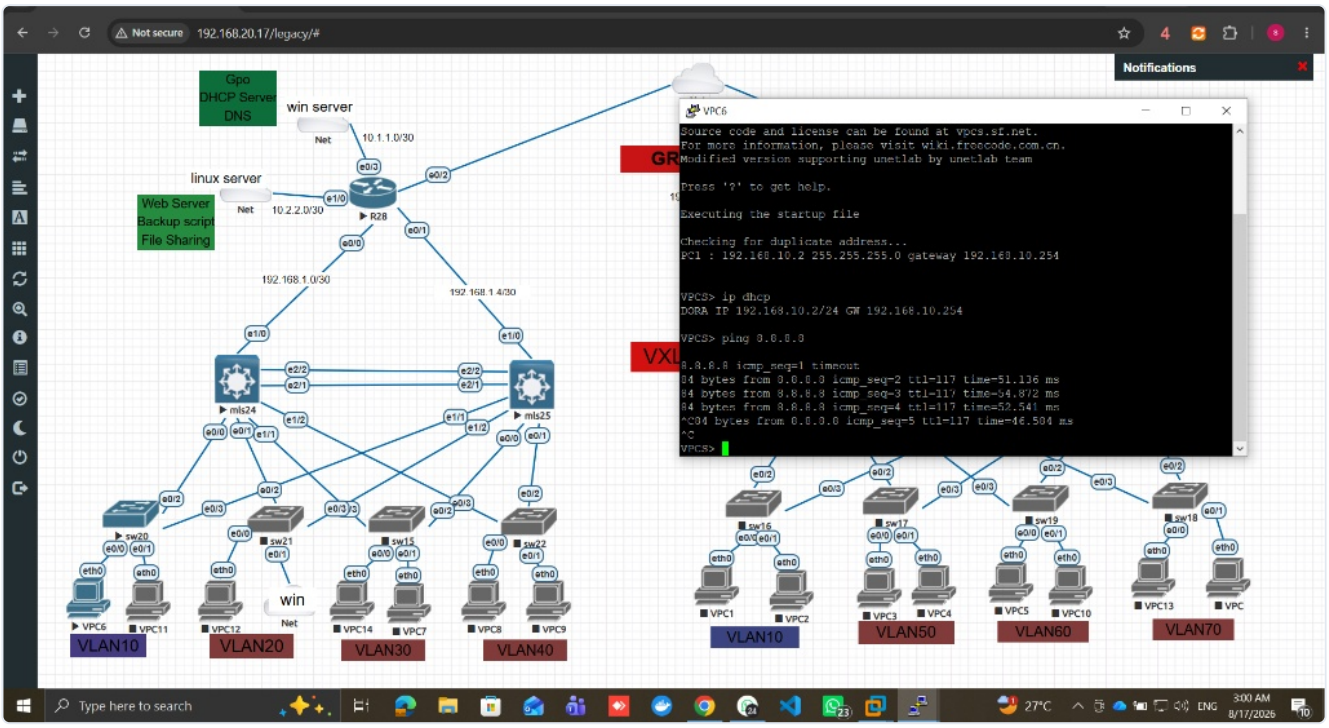

Figure 5.1: Client Terminal - DHCP Lease Acquisition (192.168.10.2) and Outbound Ping Verification (8.8.8.8)

5.2 Fault Tolerance & Error Logging Verification

When target node `r2` (2.2.2.2) experienced a connection timeout, the automation engine gracefully caught the error and recorded an entry into `ssh_failures.log`.

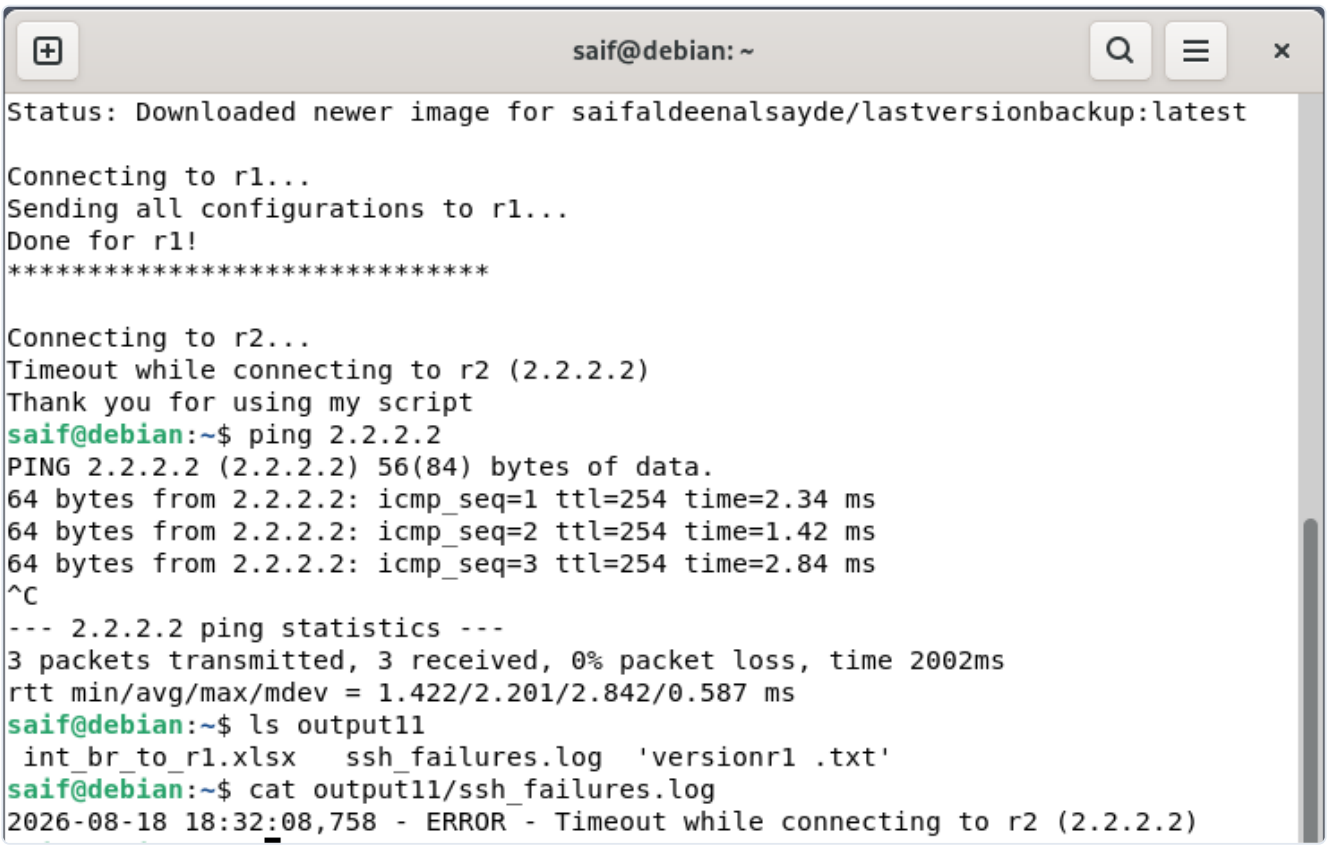

Figure 5.2: Error Handling Verification - Logging Timeout Errors into `ssh_failures.log`

5.3 Full Successful Automation Run

Once connectivity across all nodes was restored, the containerized script executed cleanly across all target routers.

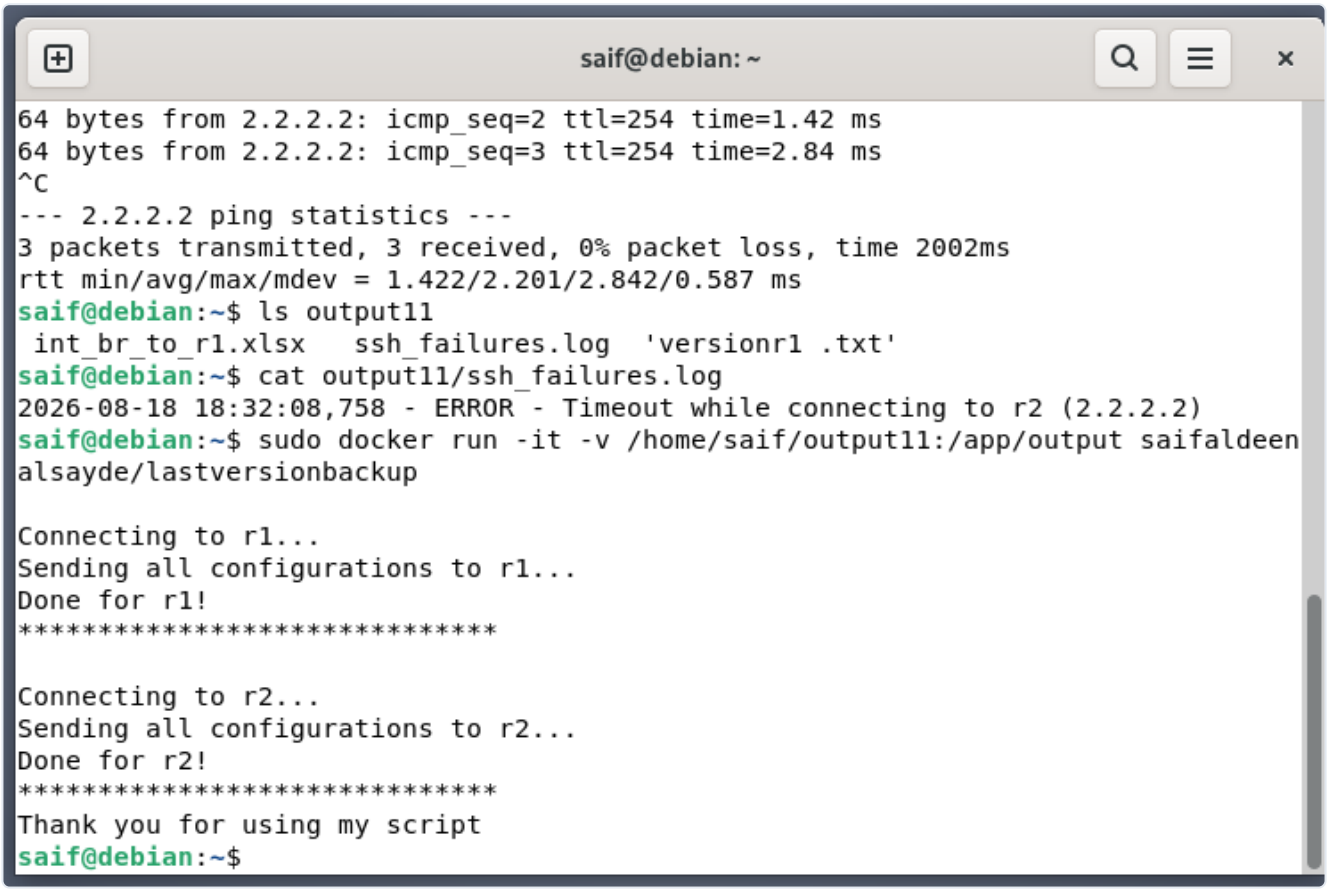

Figure 5.3: Successful Automation Pipeline Execution across Network Devices

6. Conclusion

> This project successfully validates an enterprise hybrid infrastructure integrating Cisco network routing/switching, Microsoft Active Directory domain services, Linux web hosting, and containerized Python network automation. All functional goals—including multi-scope DHCP delivery, domain membership, GRE site tunneling, and zero-touch configuration backups—were fully achieved and verified.